



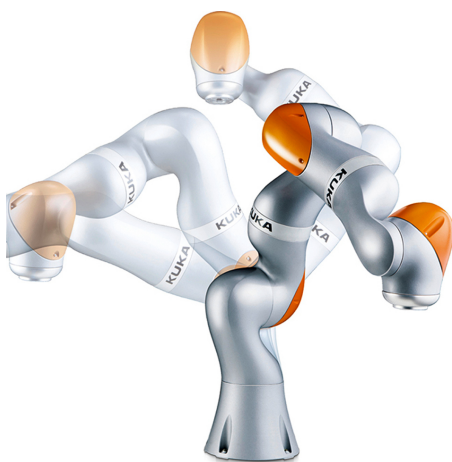
تمرین شماره ۲

۱ سینماتیک مستقیم ربات KUKA iiwa

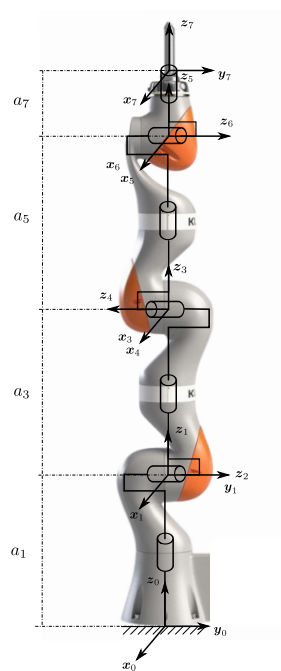
ربات هفت درجه آزادی Kuka IIWA نشان داده شده در شکل ۱ را در نظر بگیرید.

- محورهای دوران این ربات را تعیین نموده و پارامترهای دناویت-هارتنبرگ (DH) آن را استخراج کنید.
- با نوشتن یک کد برنامه‌نویسی مستقل (مبتنی بر پارامترهای DH به دست آمده)، سینماتیک مستقیم ربات را محاسبه کنید.

راهنمایی: برای حل این سؤال، از شما انتظار می‌رود دانش خود را محدود به مطالب و مراجع درس نکرده و با کمی تحقیق در منابع معتبر، به تنظیمات دقیق و پاسخ خواسته شده دست یابید.



(ب) ربات در حال انجام مانور حرکتی



(آ) ربات در وضعیت کاملاً ایستاده

شکل ۱: ربات هفت درجه آزادی Kuka IIWA

۲ شبیه‌سازی سناریوی صنعتی سیستم بازرسی خودکار

شما به تازگی به عنوان مهندس تازه‌کار رباتیک-بینایی (Junior Vision-Robotics Engineer) در یک کارخانه کوچک مونتاژ قطعات الکترونیکی استخدام شده‌اید. مهندس قبلی یک بازوی PRR (شامل یک مفصل کشویی و دو مفصل دورانی)

را به جا گذاشته است که روی یک میز کار مسطح برای بازرسی قرار دارد. این ربات هنوز کنترل کننده حلقه- بسته (Closed-loop controller) ندارد؛ در نتیجه مفاصل آن تنها در حالت سینماتیک مستقیم حلقه- باز (Open-loop forward kinematics) فرمان می گیرند.

قطعاتی که به میز کار می رسند با دیسک های نشانگر (Fiducial discs) رنگی علامت گذاری شده اند (قرمز = معیوب، سبز = اولویت دار، آبی = استاندارد). یک دوربین RGB که با زاویه ای مایل در بالا نصب شده، میز کار را مشاهده می کند. چهار نشانگر کالیبراسیون مشکی در گوشه های فضای کاری قرار دارند تا امکان اصلاح پرسپکتیو (Perspective rectification) فراهم شود.

وظیفه شما ارائه یک پایپ لاین (Pipeline) کامل و انتها به انتها است که:

۱. (بینایی): تصویر مایل دوربین را اصلاح کرده، دیسک های رنگی را بخش بندی (Segment) نموده و مرکز هر دیسک را در دستگاه مختصات پایه ربات (Robot base frame) گزارش کند.

۲. (سینماتیک): سینماتیک مستقیم نمادین بازوی PRR را از پارامترهای DH استخراج کرده و آن را به صورت عددی اعتبار سنجی کند.

۳. (شبیه سازی): این سلول کاری را در PyBullet بازسازی کرده، ربات را در مجموعه ای از پیکربندی های مفصلی مشخص شده حرکت دهد و برای هر پیکربندی پاسخ دهد که افکتور نهایی (End-effector) دقیقاً بالای کدام دیسک شناسایی شده (در صورت وجود) قرار گرفته است.

در ادامه به سوالات زیر پاسخ دهید.

بخش A — پیاده سازی سیستم بینایی ماشین (OpenCV)

دوربینی که در بالای میز کار ربات نصب شده، مستقیماً و عمود به پایین نگاه نمی کند، بلکه زاویه دار است. به همین دلیل، تصویری که به دست ما می رسد دچار اعوجاج پرسپکتیو (Projective / Keystone distortion) شده و کمی هم نویز ملایم گوسی (Gaussian noise) دارد. روی این میز کار، چهار نشانگر مربعی و مشکی رنگ در گوشه ها قرار داده شده اند که یک فضای کاری مربعی به ابعاد دقیق 0.6×0.6 متر را مشخص می کنند. در داخل این فضا، چند دیسک رنگی به عنوان قطعات هدف قرار گرفته اند.

قرارداد مختصات در این مسئله

اگر از گوشه بالا و سمت چپ میز کار فیزیکی شروع کنیم و در جهت عقربه های ساعت بچرخیم، موقعیت این چهار نشانگر مشکی در دستگاه مختصات پایه ربات (Robot base frame) به صورت زیر تعریف می شود (دقت کنید که همه واحدها بر حسب متر هستند):

نشانگر (Marker)	موقعیت روی میز	مختصات جهانی (x, y)
TL	بالا - چپ	$(+0.30, +0.30)$
TR	بالا - راست	$(+0.30, -0.30)$
BR	پایین - راست	$(-0.30, -0.30)$
BL	پایین - چپ	$(-0.30, +0.30)$

نکته مهم: فرض کنید پایه ربات دقیقاً در نقطه مرکزی و زیر میز کار محکم شده است. همچنین سطح میز کار، صفحه $z = 0$ در مختصات جهانی در نظر گرفته می شود.

تسک A.1 — شبیه سازی و تولید تصویر میز کار

در محیط واقعی صنعتی، شما تصویر را مستقیماً از دوربین می گیرید. اما در اینجا برای اینکه شرایط تمرین برای همه یکسان و قابل تکرار باشد، ما تصویر دوربین را با یک تابع برایتان شبیه سازی کرده ایم [۱]. کدهای این قسمت به عنوان «دراپور دوربین» عمل می کنند. لطفاً کدهای این سلول را اصلاً تغییر ندهید تا در ارزیابی تمرین مشکلی پیش نیاید.

این تصویر تولیدشده شامل موارد زیر است:

- چهار مربع مشکی در گوشه‌ها که برای کالیبره کردن استفاده می‌شوند.
- ۵ دیسک رنگی که مختصات واقعی و دقیق (Ground-truth) آن‌ها در متغیر `GROUND_TRUTH_DISCS` ذخیره شده است (کد شما نباید برای پیدا کردن دیسک‌ها تقلب کند و مستقیماً این متغیر را بخواند!).
- شبیه‌سازی زاویه دوربین، اثر تاریکی گوشه‌های لنز (Vignetting) و کمی نویز طبیعی تصویر.

تسک A.2 – شناسایی چهار نشانگر گوشه میز

هدف این قسمت این است که برنامه‌ی شما بتواند آن چهار مربع مشکی رنگ را به صورت خودکار پیدا کند. خروجی شما باید آرایه‌ای از مختصات این چهار نقطه باشد که حتماً به ترتیب ساعت‌گرد و شروع از گوشه بالا-چپ تصویر (`TL, TR, BR,`) مرتب شده باشند. این ترتیب‌بندی برای هماهنگی با آرایه موقعیت‌های جهانی (`WORLD_CORNERS_M`) در مراحل بعدی الزامی است.

راهنمایی: پیشنهاد می‌کنیم روی کانال روشنایی (`Value channel`) در فضای تصویر یک حد آستانه (`Threshold`) اعمال کنید تا تصویر دوتایی شود. سپس کانتورهای خارجی را بیابید. برای جدا کردن این ۴ نشانگر از بقیه نویزها، می‌توانید کانتورها را بر اساس مساحت و همچنین «مربع بودن» فیلتر کنید (مثلاً نسبت مساحت خود کانتور را با مساحت کوچک‌ترین مستطیلی که محیط بر آن است مقایسه کنید). در نهایت ۴ کانتور بزرگی که این شروط را ارضا می‌کنند نگه دارید.

تسک A.3 – اصلاح پرسپکتیو تصویر (تبدیل به نمای عمودی)

حالا که مختصات پیکسلی چهار گوشه را پیدا کردید و مختصات واقعی آن‌ها را هم روی میز می‌دانید، باید ماتریس هوموگرافی (`Homography`) را به دست آورید. این ماتریس قرار است تصویر مایل و زاویه‌دار دوربین را بگیرد و یک تصویر جدید 600×600 پیکسلی از نمای کاملاً بالا به پایین (`Top-down`) به شما تحویل دهد. برای این تصویر جدید استاندارد، این شروط باید برقرار باشد:

۱. نقطه مرکزی میز کار باید دقیقاً در وسط تصویر یعنی پیکسل $(300, 300)$ قرار بگیرد.
 ۲. هر ۱ متر در دنیای واقعی باید معادل با ۱۰۰۰ پیکسل در این تصویر باشد (با این مقیاس، میز ۶۰ در ۶۰ سانتی‌متری شما دقیقاً کل کادر ۶۰۰ در ۶۰۰ پیکسلی را پر می‌کند).
- شما باید ضریب تبدیل پیکسل به متر را در کدتان تعریف کنید، هوموگرافی را محاسبه کرده و برای اطمینان از صحت کارتان، یک شبکه شطرنجی (`Grid`) با فاصله‌های ۶ سانتی‌متری روی تصویر اصلاح‌شده رسم کنید.

تسک A.4 – بخش‌بندی و پیدا کردن دیسک‌ها در فضای HSV

اکنون روی تصویر اصلاح‌شده‌ای که در مرحله قبل ساختید، دیسک‌های سه رنگ (قرمز، سبز، آبی) را پیدا کرده و مختصات مرکز ثقل (`Centroid`) هرکدام را استخراج کنید. در این مرحله به چالش‌های زیر دقت کنید:

- در استوانه رنگ HSV، بازه رنگ قرمز از انتهای طیف به ابتدای آن می‌پرد (`Wrap-around`). باید این دو تکه شدن بازه قرمز را در کدهایتان لحاظ کنید.
- حتی بعد از اصلاح تصویر، هنوز کمی اثر تیرگی و سایه‌روشن لنز باقی مانده است که ممکن است روی رنگ‌ها تاثیر بگذارد.
- ممکن است دو دیسک هم‌رنگ کاملاً کنار و چسبیده به هم قرار گرفته باشند! بنابراین نباید کل پیکسل‌های یک رنگ را یک شیء واحد در نظر بگیرید؛ حتماً از الگوریتم تحلیل اجزای متصل (`Connected-component analysis`) استفاده کنید تا هر دیسک به درستی و مجزا شمرده شود.

خروجی این تابع باید لیستی از تاپل‌های $(color, (u, v))$ در فضای پیکسلی استاندارد باشد.

تسک A.5 — تبدیل مختصات به دستگاه ربات و تکمیل نهایی پایپلاین

در گام آخر، باید مختصات پیکسلی مرکز هر دیسک را با استفاده از تناسبی که در مرحله قبل تعریف کردید، به مختصات واقعی و متریک (نسبت به پایه ربات) تبدیل کنید.

سپس تمام توابعی که تا اینجا نوشتید را در یک تابع جامع به نام `vision_pipeline` جمع کنید. در انتهای این بخش، برنامه‌ی شما باید مختصاتی که محاسبه کرده‌اید را با مختصات واقعی قطعات (Ground-truth) مقایسه کند و در قالب یک جدول، خطای الگوریتم بینایی شما را بر حسب میلی‌متر گزارش دهد.

بخش B — سینماتیک مستقیم (Forward Kinematics) ربات PRR

در این بخش قصد داریم مدل حرکتی ربات بازرس را تحلیل کنیم. این ربات از نوع PRR است؛ یعنی دارای سه مفصل می‌باشد: مفصل اول از نوع کشویی یا منشوری (Prismatic) و دو مفصل بعدی از نوع لولایی یا دورانی (Revolute) هستند.

مشخصات مفصل و طول لینک‌ها:

- مفصل ۱ (کشویی): این مفصل در امتداد محور z جهانی حرکت می‌کند. دامنه حرکت (کورس) آن $q_1 \in [0, 0.50]$ متر است. زمانی که این مفصل در پایین‌ترین حالت خود ($q_1 = 0$) قرار دارد، محور مفصل دوم دقیقاً در ارتفاع 0.70 متری از سطح زمین قرار می‌گیرد. ما این ارتفاع اولیه را D_0 می‌نامیم.
- مفصل ۲ (دورانی): این مفصل حول محور z می‌چرخد و دامنه حرکتی آن کل دایره است: $q_2 \in [-\pi, +\pi]$.
- مفصل ۳ (دورانی): این مفصل نیز حول محور z می‌چرخد و دامنه حرکتی آن $q_3 \in [-\pi, +\pi]$ است.
- افکتور نهایی (End-effector): طول ابزار صفر در نظر گرفته شده است. ما نقطه تقاطع محور دورانی سوم با نوک لینک آخر را به عنوان افکتور نهایی یا نقطه ابزار در نظر می‌گیریم.

تسک B.1 — جدول پارامترهای DH

برای راحتی کار شما، جدول پارامترهای استاندارد DH برای ربات PRR به صورت زیر تدوین شده است (پارامترهای متغیر مفصل به صورت پررنگ مشخص شده‌اند):

لینک i	α_{i-1} (رادیان)	a_{i-1} (متر)	d_i (متر)	θ_i (رادیان)	نوع مفصل
۱	0	0	$D_0 + \mathbf{q}_1$	0	کشویی (P)
۲	0	L_2	0	$\mathbf{q}_2$	دورانی (R)
۳	0	L_3	0	$\mathbf{q}_3$	دورانی (R)

با توجه به اطلاعات مسئله، مقادیر ثابت عبارتند از: $D_0 = 0.70\text{m}$ و $L_2 = 0.40\text{m}$ و $L_3 = 0.30\text{m}$. یادآوری می‌کنیم که هر ماتریس تبدیل DH از ضرب چهار ماتریس پایه به شکل زیر به دست می‌آید:

$${}^{i-1}T_i = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_{i-1})$$

تسک B.2 — محاسبات نمادین با SymPy

در این قسمت باید با استفاده از کتابخانه SymPy در پایتون، ماتریس تبدیل هر لینک (یعنی T_0^1 ، T_1^2 و T_2^3) را به صورت پارامتری (نمادین) بسازید. سپس این ماتریس‌ها را در هم ضرب کرده و ساده‌سازی کنید تا به ماتریس تبدیل نهایی T_3^0 برسید. هدف این است که معادلات موقعیت افکتور نهایی (x, y, z) را بر حسب متغیرهای مفصلی به صورت دقیق استخراج کنید.

تسک B.3 — پیاده‌سازی عددی و اعتبارسنجی با NumPy

با حل معادلات بخش قبل، فرم بسته (Closed-form) برای موقعیت ربات به دست می‌آید. شما باید این فرمول‌ها را در قالب یک تابع پایتونی به نام `fk(q1, q2, q3)` پیاده‌سازی کنید که مختصات را در قالب یک آرایه بازگرداند. سپس برای

اطمینان از صحت کار، این تابع را به ازای ۶ حالت مفصلی تصادفی اجرا کنید و نتایج آن را با خروجی ماتریس نمادین (تسک B.2) مقایسه کنید. خطای این دو روش باید در حد صفر ماشین باشد.

تسک B.4 — رسم فضای کاری ربات (Reachable Workspace)

فضای کاری یعنی تمام نقاطی که نوک ابزار ربات می‌تواند به آن‌ها برسد. در این قسمت، یک شبکه از مقادیر مختلف برای q_1, q_2, q_3 بسازید (نمونه‌برداری با گام‌های مشخص). سپس مختصات فضایی هر نمونه را با تابع سینماتیک مستقیم حساب کرده و آن‌ها را در یک نمودار سه‌بعدی رسم کنید. برای اینکه درک بهتری از شکل استوانه‌ای این فضا پیدا کنید، نقاط را بر اساس ارتفاع (q_1) رنگ‌آمیزی کنید. در کنار آن، یک نمای بالا به پایین (تصویر روی صفحه xy) نیز رسم کنید تا نشان دهید که ربات یک فضای حلقوی (دونات‌شکل) را پوشش می‌دهد که مرز داخلی و خارجی آن به ترتیب با شعاع‌های $|L_2 - L_3|$ و $(L_2 + L_3)$ مشخص می‌شود.

تسک B.5 — رسیدن به قطعات با روش جستجوی سینماتیک مستقیم

حالا می‌خواهیم ربات را بالای سر هر کدام از دیسک‌هایی که در بخش اول (بینایی ماشین) پیدا کردید ببریم.

برای هر دیسک هدف در مختصات $(x_t, y_t, 0)$:

۱. تنظیم ارتفاع: ابتدا متغیر مفصل کشویی (q_1) را طوری با یک محاسبه ساده جبری حساب کنید که ربات در ارتفاع ایمن $z_{\text{safe}} = 0.05\text{m}$ بالاتر از سطح میز بایستد (استفاده از رابطه $z_E = D_0 + q_1$ تنها کار معکوسی است که مجاز هستید انجام دهید).

۲. جستجوی مختصات مسطح: حالا با استفاده از تکنیک جستجوی جامع (Brute-force grid search)، دامنه‌ی مفاصل دورانی (q_2 و q_3) را نمونه‌برداری کنید (ابتدا با گام درشت و سپس دقیق‌تر) تا زوج زاویه‌ای (q_2^*, q_3^*) را بیابید که وقتی آن‌ها را در تابع سینماتیک مستقیم قرار می‌دهید، فاصله نوک ربات تا مرکز دیسک (خطای صفحه xy) کمینه شود.

در نهایت نشان دهید که آیا الگوریتم شما توانسته ربات را با خطای کمتر از ۵ میلی‌متر بالای سر تمام دیسک‌ها ببرد یا خیر.

بخش C — شبیه‌سازی و اعتبارسنجی نهایی در PyBullet

اکنون زمان آن رسیده است که تمام کارهایی که در بخش‌های قبل انجام داده‌اید را در محیط سه‌بعدی شبیه‌سازی کرده و درستی عملکرد الگوریتم‌های خود را به صورت بصری و عددی اثبات کنید.

تسک C.1 — نوشتن مدل URDF ربات

برای اینکه پروژه شما کاملاً مستقل باشد و نیازی به آپلود فایل‌های جانبی نداشته باشید، کدهای XML مربوط به ساختار URDF ربات را مستقیماً به عنوان یک متغیر متنی طولانی (String) در سلول پایتون بنویسید [۱]. کد شما سپس این متن را در پوشه موقت (Temp) ذخیره می‌کند تا PyBullet بتواند آن را بخواند. ساختار این ربات شامل یک لینک پایه ثابت، لینک کشویی متحرک روی محور z ، دو لینک دورانی و یک مفصل ثابت مجازی برای افکتور نهایی است.

تسک C.2 و C.3 — راه‌اندازی سلول کاری و اعتبارسنجی سینماتیک

در این مرحله:

۱. به PyBullet متصل شوید، جاذبه را تنظیم کنید، و ربات و میز کار را در مبدا مختصات قرار دهید.
۲. برای هر دیسکی که در بخش A پیدا کرده بودید، یک استوانه رنگی کوچک در مختصات $(x, y, 0)$ روی میز ظاهر کنید.
۳. برای تک‌تک دیسک‌ها، زوایای مفصلی استخراج شده از الگوریتم جستجوی بخش B.5 را به ربات اعمال کنید (با تابع `resetJointState`).

۴. پس از حرکت ربات، مختصات افکتور نهایی را با استفاده از تابع `getLinkState` از شبیه‌ساز بخوانید و آن را با خروجی تابع ریاضی و تحلیلی `fk` مقایسه کنید. خطای بین دنیای ریاضی و شبیه‌ساز باید کمتر از ۱ میلی‌متر باشد!

تسک C.4 – ساخت انیمیشن از فرآیند بازرسی

برای جذابیت بصری پروژه، ربات باید از موقعیت صفر (موقعیت خانه یا Home) شروع به حرکت کند و به ترتیب بالای سر هر دیسک برود. برای اینکه پرش ناگهانی نداشته باشیم، بین هر دو موقعیت مفصلی متوالی، مقادیر مفاصل را به صورت خطی (Linear Interpolation) در ۱۶ فریم میانی میان‌یابی کنید. در هر فریم با `getCameraImage` یک عکس بگیرید و در نهایت با استفاده از کتابخانه `imageio` آن‌ها را به عنوان یک تصویر متحرک (GIF) ذخیره و نمایش دهید.

تسک C.5 – گزارش نهایی یکپارچه

در یک کادر (Figure) با استفاده از `matplotlib`، سه نمودار زیر را در کنار هم رسم کنید تا خلاصه‌ای از کل پروژه ارائه دهید:

۱. تصویر اصلاح شده میز کار که کانورها روی آن رسم شده و هدف‌های در دسترس با رنگ سبز مشخص شده‌اند.
۲. شماتیک بالا به پایین از میز کار، ناحیه حلقوی در دسترس ربات و موقعیت دیسک‌ها.
۳. نمودار میله‌ای (Bar chart) که خطای رسیدن به هر دیسک را (بر حسب میلی‌متر) نشان می‌دهد.

بخش D – سوالات تحلیلی

به سوالات زیر پاسخ دهید:

۱. از نظر هندسی، چرا ابعاد میزی که ربات می‌تواند تماماً آن را پوشش دهد با عبارت $(L_2 - L_3)$ مقیاس‌دهی می‌شود و به آن بستگی دارد؟ (حلقه در دسترس ربات بین $|L_2 - L_3|$ تا $L_2 + L_3$ قرار دارد).
۲. الگوریتم جستجویی که در بخش B.5 نوشتید برای یک دیسک دارای پیچیدگی زمانی $O(N^2)$ است. اگر روی میز ۳۰ قطعه وجود داشت، از چه روش محاسباتی یا پیش‌پردازی ارزان‌قیمتی می‌توانستید استفاده کنید تا بدون نیاز به حل سینماتیک معکوس تحلیلی، هزینه این جستجو برای همه اهداف سرشکن (Amortise) شود؟
۳. تصویری که در بخش A.1 ساخته شد، دارای یک اعوجاج پرسپکتیو ثابت بود. در یک محیط صنعتی واقعی با یک لنز واقعی دوربین، با چه نوع اعوجاج اپتیکال دیگری (مثلاً اعوجاج شعاعی لنز) مواجه هستید و برای رفع آن با ابزارهایی مانند `cv2.undistort`، این مرحله را باید کجای پایپ‌لاین بینایی خود قرار دهید؟
۴. شبیه‌ساز `PyBullet` موقعیت نهایی ربات را با تقریب کمتر از ۱ میلی‌متر نسبت به معادلات ریاضی ما تایید کرد. اگر موقع ساخت فایل URDF اشتباهی مرتکب می‌شدید، چه نوع خطاهایی باعث ایجاد «خطای آفست سیستماتیک» (Systematic offset) در خروجی `getLinkState` می‌شد؟
۵. فرض کنید دوربین را از سقف برداشته‌ایم و آن را به طور کاملاً صلب روی مفصل متحرک دوم ربات متصل کرده‌ایم (پیکربندی Eye-in-hand). فقط با کلمات توضیح دهید که چگونه می‌توانستید همین پایپ‌لاین را صرفاً با استفاده از سینماتیک مستقیم برای پیدا کردن موقعیت قطعات پیاده‌سازی کنید؟

منابع مربوط به A.1 و C.1

[1] https://github.com/MJAHMADEE/Robotics_Computer_Vision_Course/tree/main/Assignments/Assignment_2/assets